

Name:

BUID:

Workshop 5: Final Exam Review

1. Implement `val count : 'a -> 'a list -> int` so that `count x l` is the number of occurrences of `x` in `l`.
2. Implement the function from the previous problem with a single call to `fold_left`.
3. Design an ADT `type tree23` which can be used to represent trees in which every node either has two or three children.
4. Implement the function `val list_of_tree23 : 'a tree23 -> 'a list` which converts a `'a tree23` into an `'a list` by listing the elements in order.
5. Consider the following OCaml program.

```
type 'a foo = Foo of ('a foo -> 'a)
let bar (Foo f) = f
let baz x = bar x x
let out = baz ???
```

Given an expression to put in place of `???` such that `out` evaluates to `2`.

6. Consider the following OCaml program.

```
let x y =
  let x = y in
  let y x = x + 1 in
  let x y = y x in
  x y
```

- (a) What is the the type of `x`?
- (b) What is the value of `x 2`?

7.

```
<a> ::= <b> <c>
<b> ::= <b> y | x
<c> ::= <d> | <d> y
<d> ::= z <b> | z
```

- (a) Draw a parse tree for `x y z`
 - (b) Determine the shortest sentence accepted by this grammar that has multiple parse trees
8. In OCaml, there is an operators called `;` which can be used to sequence expressions.¹ Consider the following toy grammar.

```
<e> ::= <e1>;<e> | <e1>
<e1> ::= x | ( <e> )
```

- (a) Draw a parse tree for `x ; ( x ; x ) ; x`
 - (b) Write the BNF production rule for OCaml list literals (using `<e>` for the nonterminal symbol for expressions)
 - (c) Demonstrate that above grammar, with this new rule, is ambiguous
9. Determine an OCaml expression with the polymorphic type `('a, 'b) result -> (('a -> 'c) * ('b -> 'c)) -> 'c`

¹This is really only useful in the imperative fragment of OCaml.

10. Determine an OCaml expression with the polymorphic type `('a -> 'b, 'c -> 'b) result -> ('a * 'c) -> 'b` (look familiar?)

11. Consider the following expression.

```
(fun f -> fun x -> f x) (let z = 1 in fun y -> y + z)
```

Determine its (closure) values (look familiar?). Also write a semantic derivation.

12.

$$\{x : \tau_1, y : \tau_2\} \vdash \lambda z^T. xz(yz) : \top \rightarrow \top$$

(a) Determine types τ_1 and τ_2 such that the above judgment is derivable in STLC. Also provide the derivation.

(b) Are there multiple choices for τ_1 and τ_2 ? If so, give another choice and justify your answer.

13. Determine an expression in the λ -calculus that terminates using (small-step) CBN evaluation but gets stuck using (small-step) CBV evaluation.

14. Suppose we augment the rules of 320Caml so that adding a variable to a context does *not* shadow existing variables, but instead leaves the context unchanged in the case that variable is already declared. Demonstrate that this breaks preservation.

15. Determine the principal type of `fun f -> fun g -> fun x -> f (g (f x))`.

1. let rec count x l =

match l with

| [] -> 0

| y::l -> if x=y then 1 + count x l else count x l

2. let count x l = List.fold_left (fun acc y -> if x=y then 1+acc else acc) 0 l

3. type 'a tree23 =

| Leaf

| Node2 of 'a * 'a tree23 * 'a tree23

| Node3 of 'a * 'a tree23 * 'a tree23 * 'a tree23

4. let list_of_tree23 t =

match t with

| Leaf -> []

| Node2 (x, l, r) -> list_of_tree23 l @ [x] @ list_of_tree23 r

| Node3 (x, l, m, r) -> list_of_tree23 l @ list_of_tree23 m @ [x] @ list_of_tree23 r (+ x can also come before m)

5. baz (Foo (fun -> 2)) =

bar (Foo (fun -> 2)) (Foo (fun -> 2)) =

(fun -> 2) (Foo (fun -> 2)) =

2

6. let x₀ (y₀:int):int =

let x₁:int = y₀ in

int -> int

let y₁(x₁:int):int = x₁ + 1 in

let x₂(y₂:int -> int):int = y₂ x₁ in

x₂ y₁

x 2 = (let x=y in let y x = x+1 in let x y = y x in x y) 2

= let x=2 in let y x = x+1 in let x y = y x in x y

= let y x = x+1 in let x y = y 2 in x y

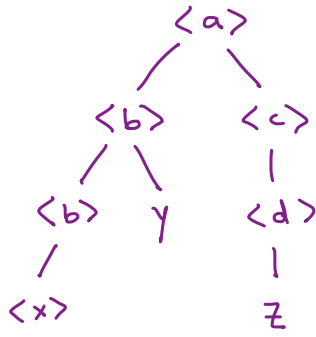
= let y = fun x -> x+1 in let x = fun y -> y 2 in x y

= let x = fun y -> y 2 in x (fun x -> x+1)

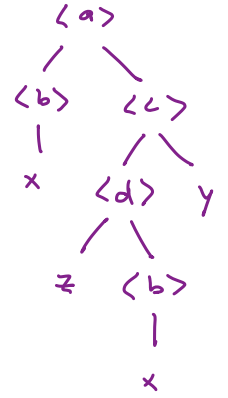
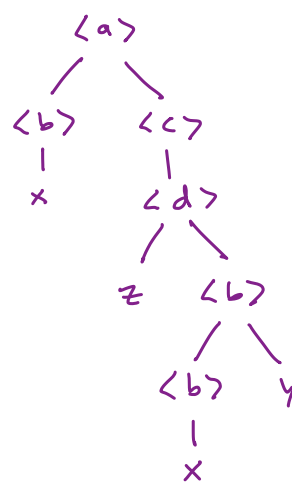
= (fun y -> y 2) (fun x -> x+1) = (fun x -> x+1) 2 = 3

Continue your work here.

7. (a)

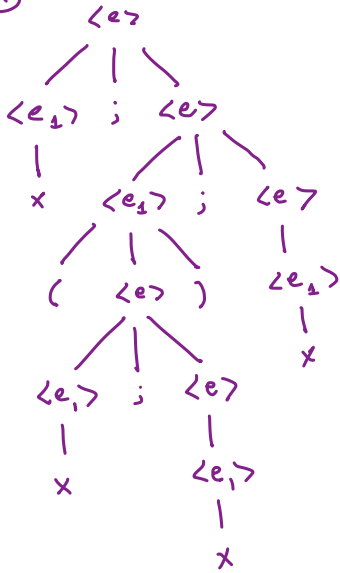


(b)



8.

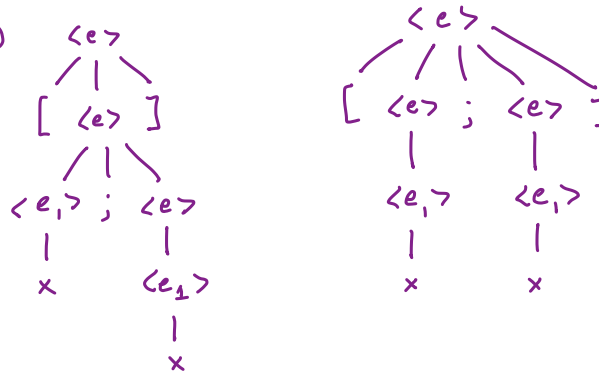
(a)



(b)

$$\langle e \rangle ::= [\langle e \rangle \{ ; \langle e \rangle \} [;]]$$

(c)



9. $\text{fun } r(f, g) \rightarrow \text{match } r \text{ with } \text{Ok } a \rightarrow f a \mid \text{Error } b \rightarrow g b$

10. $\text{fun } r(a, c) \rightarrow \text{match } r \text{ with } \text{Ok } f \rightarrow f a \mid \text{Error } g \rightarrow g c$

11. $\emptyset \vdash (\text{fun } f \rightarrow \text{fun } x \rightarrow f x) (\text{let } z = 1 \text{ in } \text{fun } y \rightarrow y + z) \quad (\text{appE})$

$\vdash \emptyset \vdash \text{fun } f \rightarrow \text{fun } x \rightarrow f x \Downarrow (\emptyset, \text{fun } f \rightarrow \text{fun } x \rightarrow f x) \quad (\text{funE})$

$\vdash \emptyset \vdash \text{let } z = 1 \text{ in } \text{fun } y \rightarrow y + z \Downarrow (\{z \mapsto 1\}, \text{fun } y \rightarrow y + z) \quad (\text{letE})$

$\vdash \emptyset \vdash 1 \Downarrow 1 \quad (\text{intLitE})$

$\vdash \{z \mapsto 1\} \vdash \text{fun } y \rightarrow y + z \Downarrow (\{z \mapsto 1\}, \text{fun } y \rightarrow y + z) \quad (\text{funE})$

$\vdash \{f \mapsto (\{z \mapsto 1\}, \text{fun } y \rightarrow y + z)\} \vdash \text{fun } x \rightarrow f x \Downarrow$

$(\{f \mapsto (\{z \mapsto 1\}, \text{fun } y \rightarrow y + z)\}, \text{fun } x \rightarrow f x) \quad (\text{funE})$

Continue your work here.

$$\begin{aligned}
 12. \quad & \textcircled{a} \quad \{x:\tau_1, y:\tau_2\} \vdash \lambda z. xz(yz) : T \rightarrow T \\
 & \quad \vdash \{x:\tau_1, y:\tau_2, z:T\} \vdash xz(yz) : T \\
 & \quad \vdash \left\{ \begin{array}{l} \{ \dots \} \vdash xz : \tau_3 \rightarrow T \\ \quad \vdash \left\{ \begin{array}{l} \{ \dots \} \vdash x : \tau_1 \\ \quad \vdash \{ \dots \} \vdash z : T \end{array} \right. \quad (= T \rightarrow \tau_3 \rightarrow T) \\ \quad \vdash \{ \dots \} \vdash yz : \tau_3 \\ \quad \vdash \left\{ \begin{array}{l} \{ \dots \} \vdash y : \tau_2 \\ \quad \vdash \{ \dots \} \vdash z : T \end{array} \right. \quad (= T \rightarrow \tau_3) \end{array} \right.
 \end{aligned}$$

$$\tau_1 = T \rightarrow \tau_3 \rightarrow T$$

$$\tau_2 = T \rightarrow \tau_3 \quad \text{for any } \tau_3$$

eg

$$\tau_1 = T \rightarrow T \rightarrow T \quad \text{and} \quad \tau_1 = T \rightarrow (T \rightarrow T) \rightarrow T$$

$$\tau_2 = T \rightarrow T$$

$$\tau_2 = T \rightarrow T \rightarrow T$$

$$13. (\lambda x. \lambda y. y) z$$

$$(\lambda x. \lambda y. y) z \xrightarrow{\text{CBN}} \lambda y. y$$

$$(\lambda x. \lambda y. y) z \not\xrightarrow{\text{CBV}} \lambda y. y$$

$$14. \text{ let } x = 1 \text{ in} \\ \text{ let } x = \text{true in} \\ x + x$$

$$\emptyset \vdash \text{let } x = 1 \text{ in let } x = \text{true in } x + x : \text{int} \quad (\text{let})$$

$$\vdash \left\{ \begin{array}{l} \emptyset \vdash 1 : \text{int} \quad (\text{intLit}) \\ \{x:\text{int}\} \vdash \text{let } x = \text{true in } x + x : \text{int} \quad (\text{let}) \end{array} \right.$$

$$\vdash \left\{ \begin{array}{l} \{x:\text{int}\} \vdash \text{true} : \text{bool} \quad (\text{true}) \\ \{x:\text{int}\} \vdash x + x : \text{int} \quad (\text{addInt}) \end{array} \right.$$

$$\vdash \left\{ \begin{array}{l} \{x:\text{int}\} \vdash x : \text{int} \quad (\text{intLit}) \\ \{x:\text{int}\} \vdash x : \text{int} \quad (\text{intLit}) \end{array} \right.$$

$$\text{let } x = 1 \text{ in let } x = \text{true in } x + x \rightarrow_p$$

$$\text{let } x = \text{true in } x + x$$

$$\emptyset \not\vdash \text{let } x = \text{true in } x + x : \text{int}$$

Continue your work here.

$$15. \quad \emptyset \vdash \lambda f. \lambda g. \lambda x. f(g(fx)) : \alpha \rightarrow \beta \rightarrow \gamma \rightarrow \eta \vdash C \text{ (fun)}$$

$$\vdash \{f:\alpha\} \vdash \lambda g. \lambda x. f(g(fx)) : \beta \rightarrow \gamma \rightarrow \eta \vdash C \text{ (fun)}$$

$$\vdash \{f:\alpha, g:\beta\} \vdash \lambda x. f(g(fx)) : \gamma \rightarrow \eta \vdash C$$

$$\vdash \{f:\alpha, g:\beta, x:\gamma\} \vdash f(g(fx)) : \eta \vdash \alpha \dot{=} \varepsilon \rightarrow \eta, \beta \dot{=} \delta \rightarrow \varepsilon, \alpha \dot{=} \gamma \rightarrow \delta \text{ (app)}$$

$$\vdash \{\dots\} \vdash f : \alpha \vdash \emptyset \text{ (var)}$$

$$\vdash \{\dots\} \vdash g(fx) : \varepsilon \vdash \beta \dot{=} \delta \rightarrow \varepsilon, \alpha \dot{=} \gamma \rightarrow \delta \text{ (app)}$$

$$\vdash \{\dots\} \vdash g : \beta \vdash \emptyset \text{ (var)}$$

$$\vdash \{\dots\} \vdash f x : \delta \vdash \alpha \dot{=} \gamma \rightarrow \delta \text{ (app)}$$

$$\vdash \{\dots\} \vdash f : \alpha \vdash \emptyset \text{ (var)}$$

$$\vdash \{\dots\} \vdash x : \gamma \vdash \emptyset \text{ (var)}$$

$$\alpha \dot{=} \varepsilon \rightarrow \eta \text{ (A)} \quad S = [\alpha \mapsto \varepsilon \rightarrow \eta, \beta \mapsto \delta \rightarrow \varepsilon, \varepsilon \mapsto \gamma, \eta \mapsto \delta]$$

$$\beta \dot{=} \delta \rightarrow \varepsilon \text{ (A)}$$

$$\alpha \dot{=} \gamma \rightarrow \delta \text{ (F)}$$

$$\varepsilon \rightarrow \eta$$

$$\varepsilon \dot{=} \gamma \text{ (A)}$$

$$\eta \dot{=} \delta \text{ (A)}$$

$$\alpha \rightarrow \beta \rightarrow \gamma \rightarrow \eta$$

$$\downarrow \alpha \mapsto \varepsilon \rightarrow \eta$$

$$(\varepsilon \rightarrow \eta) \rightarrow \beta \rightarrow \gamma \rightarrow \eta$$

$$\downarrow \beta \mapsto \delta \rightarrow \varepsilon$$

$$(\varepsilon \rightarrow \eta) \rightarrow (\delta \rightarrow \varepsilon) \rightarrow \gamma \rightarrow \eta$$

$$\downarrow \varepsilon \mapsto \gamma$$

$$(\gamma \rightarrow \eta) \rightarrow (\delta \rightarrow \gamma) \rightarrow \gamma \rightarrow \eta$$

$$\downarrow \eta \mapsto \delta$$

$$S \uparrow = (\gamma \rightarrow \delta) \rightarrow (\delta \rightarrow \gamma) \rightarrow \gamma \rightarrow \delta$$

$$\forall \alpha. \forall \beta. (\alpha \rightarrow \beta) \rightarrow (\beta \rightarrow \alpha) \rightarrow \alpha \rightarrow \beta$$